Holistic Evolution of AI-Driven Pedagogical Frameworks: A Technical and Empirical Evaluation of the InterviewIQ Mock Interview Platform

Rakesh Kumar

Enrollment No: SBU2401576 | MCA Final Year

Project Supervisor: Dr. S. K. Modak

Assistant Professor, Dept. of CSE, Sarala Birla University, Ranchi

May 4, 2026

Abstract

Campus placement drives bring a familiar anxiety every year. Students who have studied hard suddenly find themselves unable to string together a coherent answer when an interviewer is actually watching them. This disconnect — between knowing the subject and performing under observation — is not about intelligence. It is mostly about practice, and the right kind of practice is genuinely hard to get. This paper reports on *InterviewIQ*, a browser-based SaaS platform we developed at Sarala Birla University to address this specific problem. The platform combines LLM-driven question generation, SBERT-based resume analysis, and real-time voice processing to simulate technical job interviews. In a structured pilot with 40 MCA students, we measured a 23.1% gain in structured answer quality and an 84.8% rise in self-confidence scores compared to a control group that prepared through conventional methods. Feedback latency dropped from 48–72 hours to under two seconds. Perhaps more interesting than the numbers: students in the treatment group reported that the absence of human judgment made them willing to attempt answers they would have otherwise skipped entirely. The implications of that, for how we think about deliberate practice in technical education, are worth exploring.

1. Introduction

I should start with a confession. When this project began, I assumed the hard part would be the AI integration. OpenAI APIs, embeddings, streaming — that is the kind of work that sounds difficult and, in practice, has decent documentation. The genuinely hard part turned out to be something much less glamorous: figuring out why students were not using the system even after we built it.

The answer, after some informal conversations with early testers, was simple. They were embarrassed. Even in a private browser session with no human observer, they did not want to give a wrong answer to a system that was recording them. This told us something important about the psychology of interview preparation — and it eventually shaped the design of the feedback interface more than any technical consideration.

That is the context for InterviewIQ. The platform is a response to a structural problem in how MCA students prepare for placement. Quality mock interview partners are scarce. Paid coaching is expensive, often beyond what students at Tier 3 institutions can access. Peer practice is inconsistent — the student playing "interviewer" typically does not know the domain well enough to push back on weak answers. These are not new problems. But the arrival of capable LLMs in 2023-24 made a software solution genuinely feasible for the first time.

The theoretical grounding we worked with is Kolb's Experiential Learning Cycle [1, 2], which argues that learning deepens through a loop of doing, observing, reflecting, and adjusting. Traditional interview prep breaks this loop at the feedback stage — the reflection either does not happen, or happens too late to be useful. Our hypothesis was that sub-two-second AI feedback could close this loop tightly enough to produce real learning gains. That hypothesis held up. Section 6 has the data.

The rest of this paper is structured as follows. Section 2 describes the system architecture. Section 3 covers the LLM orchestration layer. Section 4 explains the voice processing pipeline. Section 5 covers SaaS operations. Section 6 presents our empirical findings. Section 7 closes with limitations and what we would do differently.

2. System Architecture and Technical Design

Choosing a tech stack for a student project is always a negotiation between what would be ideal and what you can actually debug at 2am before a submission deadline. We made some choices that were technically optimal and a few that were pragmatic. Both kinds are worth documenting.

The system runs on two separate backend services. Real-time API orchestration — user sessions, authentication, streaming responses — runs on *Node.js* with Express. Computationally heavier tasks, specifically model inference and document processing, go through a *Python-based FastAPI* service. This split was initially accidental: the resume parsing code was written in Python because that is where the good libraries are, and we never moved it. As it turned out, the separation was beneficial — it let us scale the inference service independently during load testing without touching the session management layer.

2.1 Authentication and Data Isolation

We use **JWT-based authentication with Role-Based Access Control (RBAC)**. For a multi-tenant SaaS product handling personal data — resumes, audio transcripts, session histories — data isolation is not optional. Each user's data must be strictly scoped to their account, and admin-level routes must be protected separately.

The persistence layer uses MongoDB. The choice of a NoSQL store was deliberate: interview session data is unstructured in ways that are difficult to predict at schema design time. Sessions vary in length, question count, response format, and domain. A rigid relational schema

would have required either constant migrations or over-engineered flexibility. MongoDB let us store session documents as they naturally occurred and query them later.

One thing we underestimated: JWT verification latency under concurrent load. When 15+ users were running sessions simultaneously in testing, token verification was adding around 80ms per request — perceptible in a real-time streaming context. We solved this by caching verification results with a short TTL and switching to a refresh-token pattern for sustained sessions. The fix worked, but it added complexity to the client we had not planned for.

2.2 Resume Parsing and Semantic Matching

Generic questions do not help anyone prepare for a specific role. If you are applying for a data engineering position, being drilled on general software engineering questions is only marginally useful. Personalisation, therefore, was a non-negotiable feature from the start, not something added later.

Resumes arrive as PDF files. We extract text using *pdfplumber* [5], which handles the messy layout of most real-world CVs acceptably well. I say 'acceptably' because there were formatting edge cases — multi-column layouts, embedded tables, scanned documents — that pdfplumber handled poorly. For the pilot, we added a plain-text fallback that prompted users to paste their resume content if the parsed output looked malformed. About 12% of users hit this fallback.

Candidate-to-role matching uses **Sentence-BERT embeddings** (all-mpnet-base-v2 variant). The similarity score between a resume embedding (E_r) and a job description embedding (E_j) is computed as cosine similarity:

$$Similarity(E_r, E_j) = (E_r \cdot E_j) \;/\; (\|E_r\| \times \|E_j\|)$$

Empirical accuracy on our test set was around 71% [5]. That is functional for a first version. The main failure mode we observed was domain-switching candidates — a mechanical engineering student applying for software roles — where the embedding distance is large but the candidate is actually qualified. We do not have a clean solution for this yet. It is logged as a known limitation.

3. LLM Orchestration Layer

The LLM integration is where the project spent most of its iteration cycles. Getting a language model to generate a useful interview question is easy. Getting it to generate a question that is appropriately calibrated to this candidate's experience level, on this specific topic, in a way that builds naturally on the previous exchange — that is considerably harder.

3.1 Model Routing via OpenRouter

We route all model calls through the **OpenRouter API** [6, 7]. The primary motivation was risk management: if any single provider has a service disruption or changes its pricing significantly,

we can switch without rewriting the inference layer. In practice during the pilot, we ran DeepSeek-R1 for most question generation tasks and Claude 3 Opus for evaluation feedback. The two models have meaningfully different strengths, and having both available via a single interface was valuable.

One operational detail that is worth flagging: OpenRouter's behaviour changes noticeably when account balance drops below approximately $10. Additional database checks kick in on their end, and the Time to First Token (TTFT) can spike from ~300ms to over a second [8, 9]. For a real-time interview simulation, that latency difference is very noticeable to users. We treat maintaining credit balance as a performance requirement now, not just a billing task.

Table 1. Benchmark Comparison of LLM Models Used in InterviewIQ (2026)

Model	Primary Use	Context Window	GPQA Score
DeepSeek-R1	Question Generation	64,000 tokens	70.8%
Claude 3 Opus	Formative Feedback	200,000 tokens	95.4%
GPT-5.5	Complex Reasoning	1,000,000 tokens	93.6%
OpenAI o3-mini	Low-Latency Tasks	200,000 tokens	85%+

Source: OpenRouter documentation and published model benchmarks [6, 7]

3.2 Token Streaming with Server-Sent Events

We use **Server-Sent Events (SSE)** for streaming model responses token by token to the client [10, 11]. We tested WebSockets first. The stateful connection management was more complexity than the use case warranted — SSE does the same job with standard HTTP infrastructure and no persistent connection state to manage.

The user-facing impact of streaming is larger than I expected. Even when total response time is identical, incremental display feels dramatically more responsive. In user interviews, several students described the streaming display as making the AI feel "like it is actually thinking," which, whether or not that is technically accurate, meaningfully improved session engagement.

4. Voice Processing Pipeline

Moving from text-based Q&A to spoken dialogue was the feature that took longest to stabilise. Not because the core technology is complicated — the Web Speech API is well-documented — but because browser implementations are inconsistent in ways that are not documented anywhere useful.

The voice pipeline has two parts. **Automatic Speech Recognition (ASR)** transcribes the candidate's spoken responses in real time using the browser's native speech recognition [12]. We wrote separate handling code for Chrome and Firefox because interim result behaviour differs between them. Without this, Firefox users saw their responses appear only after they stopped speaking, which broke the live feedback feel entirely.

On the output side, **Text-to-Speech (TTS)** converts AI-generated questions into spoken audio [13, 14]. The main technical issue here was browser autoplay policy: modern browsers block audio that starts without a user gesture. We lost two days to this before implementing an explicit AudioContext unlock triggered on the first user interaction. Simple fix, non-obvious to discover.

We did not build custom speech models. The Web Speech API quality is sufficient for clear speech in reasonably quiet environments. It degrades on heavy regional accents and technical jargon — two things that come up frequently in engineering interviews. This is a real limitation, and we flag it as an area needing improvement before any wider deployment.

5. SaaS Infrastructure and Financial Operations

5.1 Payment Processing and Reconciliation

The platform uses a credit-based consumption model. Users purchase session credits, which are deducted per interview session. Payment processing is handled by **Razorpay**, which integrates cleanly with Indian payment systems including UPI.

The interesting engineering problem is not the payment itself — it is reconciliation [15]. A user initiates payment, Razorpay processes it, and a webhook fires to our server. But webhooks can fail. They can be delayed, duplicated, or silently dropped. Without a reconciliation layer, you end up with situations where a user paid successfully but their credits were never allocated, or — worse — credits were allocated twice.

We built an Automated Reconciliation Service that periodically cross-checks Razorpay transaction records against our internal credit ledger. Any discrepancy triggers a manual review flag rather than automatic correction. In three months of operation, we caught four genuine mismatches that would otherwise have gone unnoticed. Two were un-credited payments; two were potential double-allocations. The manual review requirement felt conservative at design time; it felt exactly right after we saw what it caught.

5.2 Administrative Oversight and Ethical Considerations

An admin dashboard with isAdmin-gated routes lets us monitor platform activity and review flagged sessions. The ethical framing here matters. The system records spoken responses and transcribes them. Users consent to this, but that consent creates an obligation to handle the data carefully and to provide meaningful oversight of what the AI is doing with it.

We observed during the pilot that the AI occasionally gave feedback that was technically accurate but phrased in ways that could be discouraging — particularly for students who were clearly struggling. Human reviewers flagged three such sessions in the first month. We subsequently added a tone calibration instruction to the feedback prompt. The number of negative flags dropped to zero in the following month. This is exactly the kind of correction that requires a human-in-the-loop to catch.

6. Empirical Results and Analysis

We ran a controlled pilot study over six weeks. Forty MCA students were divided into two groups of twenty. Group A used InterviewIQ for structured practice sessions — a minimum of three sessions per week. Group B prepared through conventional methods: peer mock interviews, online resources, and self-study. Both groups were assessed by the same panel of evaluators, who were blind to which group each student belonged to.

The main findings are summarised in Table 2.

Table 2. Performance Metrics: Conventional Preparation vs. InterviewIQ (n=40)

Metric	Control Group	InterviewIQ Group	Difference
Structured Answer Quality	62.4%	76.8%	+23.1%
Self-Reported Confidence	45.0%	83.2%	+84.8%
Filler Word Frequency	15.0%	4.0%	−66.0%
Average Feedback Latency	48–72 hours	<2 seconds	−99.9%

Source: Pilot study conducted at Sarala Birla University, January–March 2026

The answer quality improvement (23.1%) was roughly what we had predicted based on the literature on deliberate practice [16, 17]. The confidence jump (84.8%) was not. We had expected some improvement, but not of that magnitude.

Post-study interviews helped explain it. Repeatedly, students mentioned that knowing there was no human observer made them willing to attempt answers they would otherwise have avoided. One student described it as: "I can say something stupid and figure out why it is stupid, without anyone remembering that I said it." That is a real form of psychological safety, and it appears to have a measurable effect on learning behaviour.

An important nuance: students did not say they preferred AI feedback to human feedback. Most still rated human feedback as more valuable when they could access it. What InterviewIQ seemed to do was get students to a baseline level of fluency that made subsequent human feedback sessions more productive. This framing — AI preparation as a complement to human mentorship, not a replacement — seems like the honest and useful way to position the platform.

78% of Group A students reported that AI-driven sessions felt less intimidating than human mock interviews. Whether that is entirely a virtue is an open question. Some degree of pressure is probably useful preparation for actual interviews, which are stressful. Finding the right calibration between psychological safety and realistic stress is something we want to investigate in the next study phase.

7. Conclusions and Future Directions

InterviewIQ works. That is the honest summary of what we built and what the pilot data showed. It is not a perfect system — the SBERT matching at 71% accuracy leaves room for improvement, the voice pipeline struggles with accents and technical jargon, and the question quality in very specialised domains like embedded systems or compiler design is weaker than in mainstream web development topics. We know this because students told us, specifically and clearly.

What the system does well is close the feedback latency gap that makes traditional interview preparation inefficient. Getting feedback in under two seconds instead of 48 hours changes how students engage with the practice. The data on confidence gains, in particular, suggests that the platform's value may be as much psychological as pedagogical — giving students a space to fail safely, repeatedly, until the material becomes genuinely automatic.

Two directions are most interesting to us going forward. The first is **multimodal analysis** — incorporating non-verbal cues like eye contact, filler gestures, and posture into the feedback loop. Computer vision for these tasks is technically feasible. The harder design problem is doing it without the system feeling like a surveillance tool, which would destroy the psychological safety that makes it useful [18, 19].

The second is adaptive difficulty. Right now, question difficulty is calibrated once at session start based on resume domain. A system that tracks real-time performance within a session and adjusts question complexity accordingly would be more effective — and more interesting to build. We are currently designing the architecture for this.

On the bigger picture: the gap this project addresses is structural and not going away. Elite interview coaching has always existed; what InterviewIQ attempts is to make a version of it accessible to students who cannot afford the paid alternative. Whether scalable AI tools can genuinely democratise professional preparation, or whether they simply create a new tier of advantage for students who know how to use them, is a question worth tracking carefully as this technology matures.

References

[1] ResearchGate. (2025). AI-Powered Mock Interview System for Automated Skill Assessment. ResearchGate Publications.

[2] IJRASET. (2025). AI-Powered Mock Interview System. International Journal for Research in Applied Science and Engineering Technology, Vol. 13(11).

[3] ResearchGate. (2026). Real-Time AI Feedback for Performance-Based Learning.

[4] Wilson, A.D. & Golonka, S. (2013). Embodied Cognition is Not What you Think it is. Frontiers in Psychology, 4, 58.

[5] JATIT. (2025). AI-Powered Resume Screening System using Sentence-BERT Embeddings. Journal of Applied Theoretical and Information Technology, Vol. 103(23).

[6] OpenRouter. (2025). DeepSeek-R1 API Benchmarks and Reasoning Capabilities. openrouter.ai.

[7] OpenRouter. (2026). Model Catalogue and Performance Benchmarks. openrouter.ai/docs.

[8] OpenRouter Docs. (2026). Latency and Performance Best Practices for Production Deployments.

[9] Procedure Tech. (2025). The Streaming Backbone of LLMs: Why SSE Wins Over WebSockets.

[10] W3C. (2026). Server-Sent Events Specification. World Wide Web Consortium.

[11] Fielding, R.T. (2000). Architectural Styles and the Design of Network-Based Software Architectures. Doctoral Dissertation, UC Irvine.

[12] MDN Web Docs. (2026). Web Speech API Reference. Mozilla Developer Network.

[13] W3C. (2026). Web Speech API Specification for Browser Security Compliance.

[14] freeCodeCamp. (2026). Building Voice AI Applications with the Web Speech API.

[15] Dodo Payments. (2026). Payment Reconciliation Algorithms for SaaS Finance.

[16] Baker, E.L. (2021). Corrective Instructions in Performance-Based Skill Execution.

[17] MDPI Education Sciences. (2026). Generative AI as a Pedagogical Agent in Experiential Learning Frameworks.

[18] Sharma, R. & Singh, P. (2025). AI-Powered Mock Interview Platform Using Computer Vision for Non-Verbal Analysis.

[19] Chou, Y. et al. (2024). AI-Based Interview Evaluation: Emotion Recognition and Confidence Scoring Models.